

# CYBERSTER

Purple Team Internship Program | Batch 1

---



## Week 1 Lab Report

*Network Security & Protocol Analysis*

**Muhammed Umer Khan**

Roll No: CSI-B1-597

Submission Date: March 8, 2026

Track: Purple Team | Duration: Week 1

---

---

## ■ Executive Summary

This report documents the successful completion of Week 1 tasks within the Cyberster Purple Team Internship Program (Batch 1). The objective was to establish a fully isolated virtual lab environment using VMware Workstation and to perform foundational network packet analysis using Wireshark.

Two virtual machines were configured — Kali Linux 2025.4 as the attacker node (192.168.56.30) and Windows 11 as the target (192.168.56.40) — connected via an isolated LAN Segment named “Intnet” with zero external internet access. Wireshark was used to capture a live TCP Three-Way Handshake triggered by an Nmap port 445 scan, and the Ethernet, IP, and TCP protocol headers were individually analyzed for forensic significance.

## ■ Lab Environment Overview

| Component           | Details                                                      |
|---------------------|--------------------------------------------------------------|
| Hypervisor          | VMware Workstation                                           |
| Attacker VM         | Kali Linux 2025.4 (amd64) — kali / kali                      |
| Target VM           | Windows 11 (Build 10.0.26200.7840)                           |
| Network Mode        | LAN Segment “Intnet” — fully isolated (no DHCP, no internet) |
| Kali IP / Interface | 192.168.56.30 / 255.255.255.0 on eth0                        |
| Windows 11 IP       | 192.168.56.40 / 255.255.255.0 on Ethernet0                   |
| Packet Capture Tool | Wireshark (launched on Kali Linux)                           |
| Traffic Generator   | Nmap v7.95 — TCP port 445 scan                               |

## ■ Task 01 — Virtualization & Network Topology Setup

### ■ Objective

To build a secure, isolated Purple Team lab where Kali Linux (attacker) and Windows 11 (target) communicate privately with zero external internet access — mirroring a real-world controlled security testing environment.

### ■ 1.1 LAN Segment Configuration in VMware Workstation

Both VMs were fully powered off. In VMware Workstation → VM Settings → Network Adapter, the connection type was changed from NAT to LAN Segment. A new segment named “Intnet” was created, and both VMs were assigned to the same segment to create a private, isolated virtual switch with no gateway or DHCP.

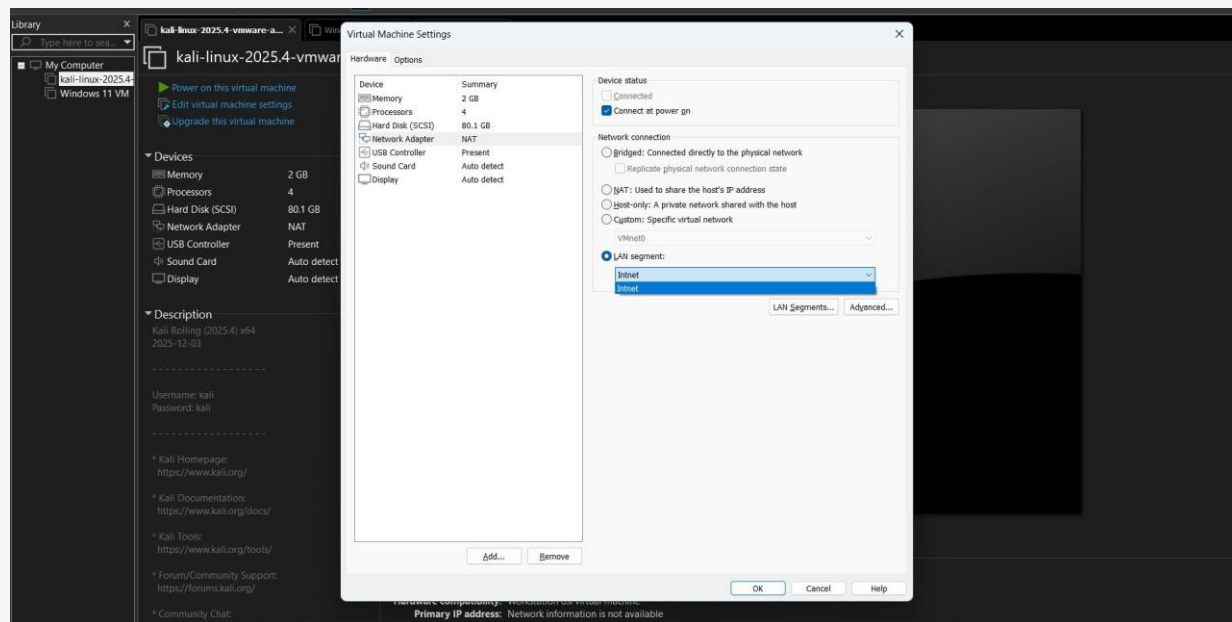


Figure 1 — Kali Linux VM Settings: Network Adapter switched to LAN Segment “Intnet”

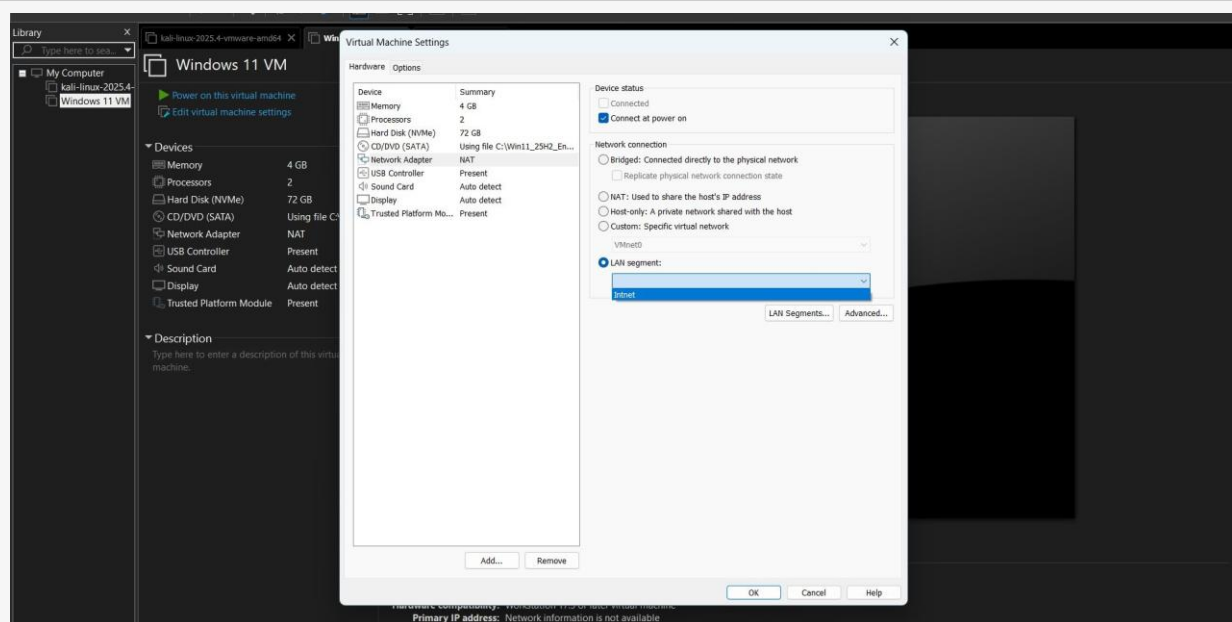


Figure 2 — Windows 11 VM Settings: Network Adapter set to the same LAN Segment “Intnet”

## 1.2 Static IP Assignment on Kali Linux

Since LAN Segments have no DHCP server, static IPs must be configured manually. Kali Linux was first checked with `ip addr` — no IPv4 address was present on `eth0`. The Network Manager GUI was then opened and the IPv4 Settings tab was set to Method: Manual with Address 192.168.56.30 and Netmask 255.255.255.0. After saving, the NetworkManager service was restarted and the assignment verified with `ip addr show`.

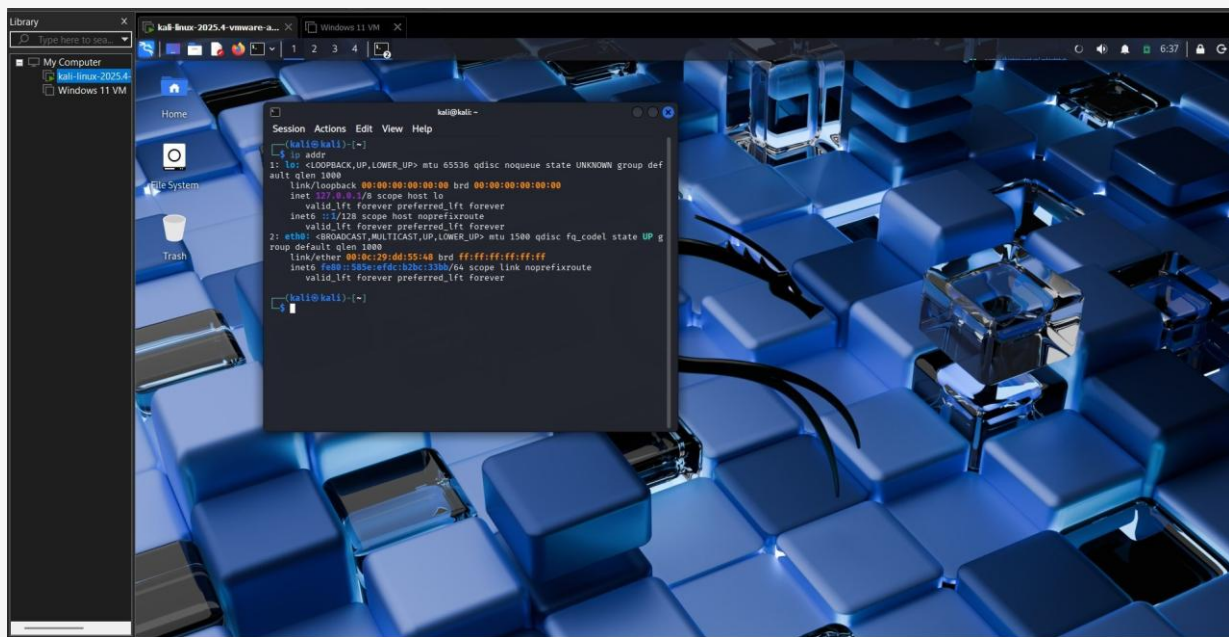


Figure 3 — Kali Linux: `ip addr` showing no IPv4 address on `eth0` before static IP assignment

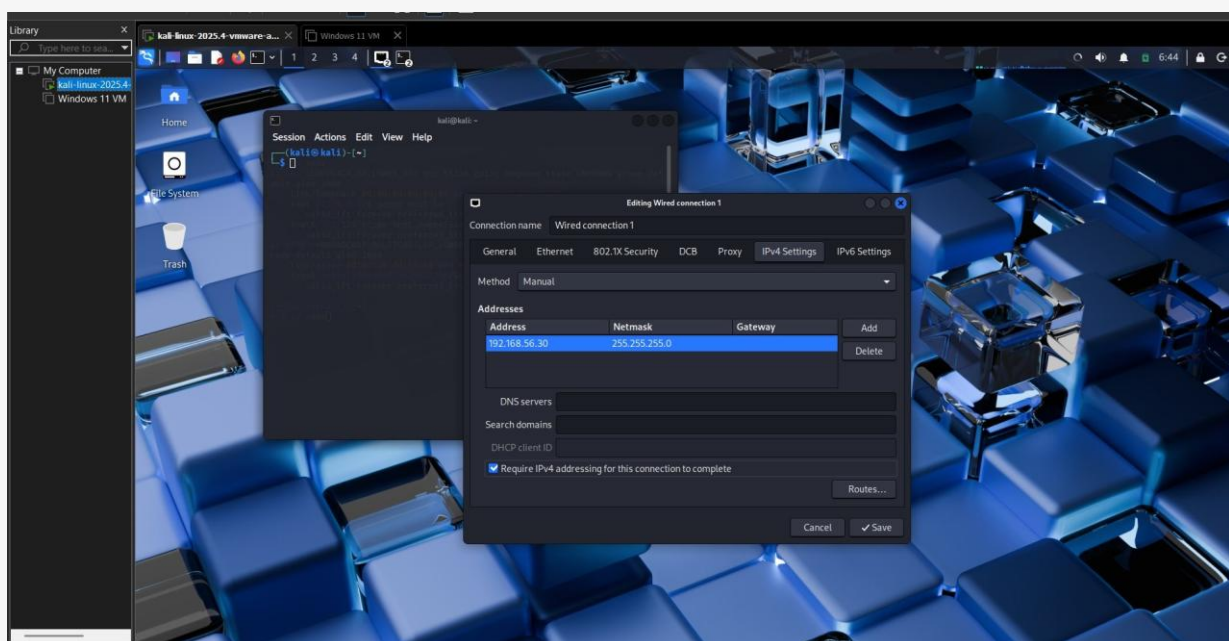
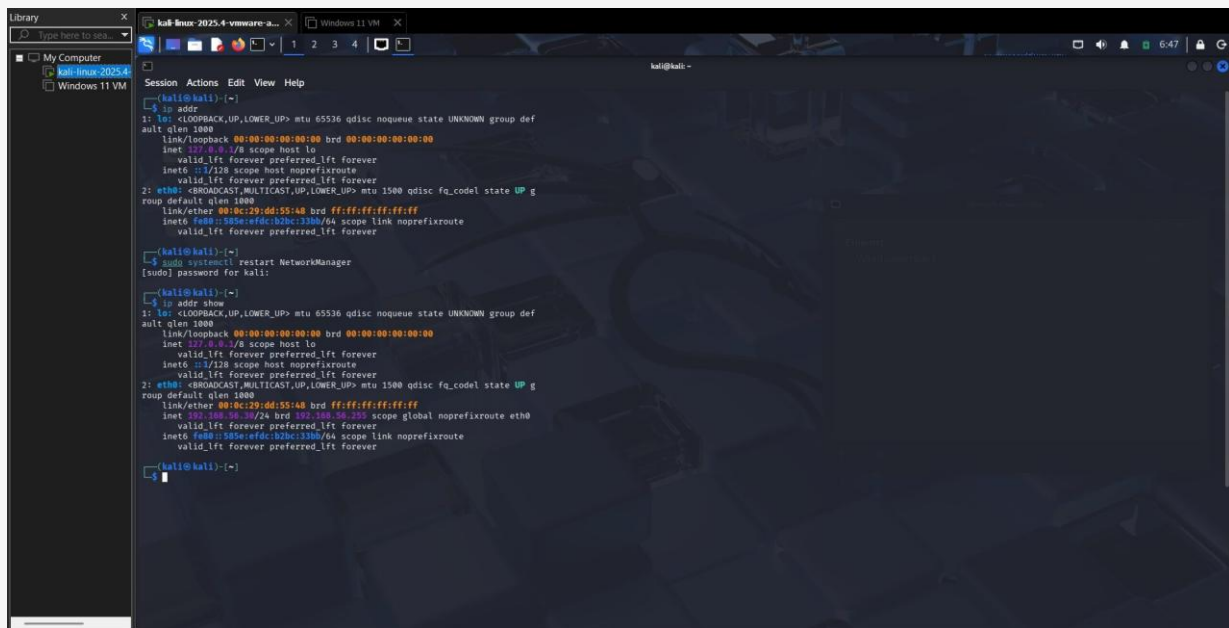


Figure 4 — Kali Linux: Network Manager GUI — IPv4 Settings set to Manual with 192.168.56.30



```
kali@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
    link/ether 00:0c:29:d4:55:a8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.40/24 scope link noprefixroute
        valid_lft forever preferred_lft forever

kali@kali:~$ ip netns exec ns1 ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
    link/ether 00:0c:29:d4:55:a8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.40/24 scope link noprefixroute
        valid_lft forever preferred_lft forever
    inet6 fe80::505e:efdc:b2bc:338a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure 5 — Kali Linux terminal: `ip addr show` confirming 192.168.56.30/24 on `eth0` after assignment

### 1.3 Static IP Assignment on Windows 11

On Windows 11, the IP was configured via Settings → Network & Internet → Ethernet → Edit IP assignment. The method was changed to Manual, IPv4 was toggled on, and IP 192.168.56.40 with Subnet 255.255.255.0 was entered and saved. Windows Defender Firewall was disabled for all profiles to permit ICMP ping traffic during lab verification.

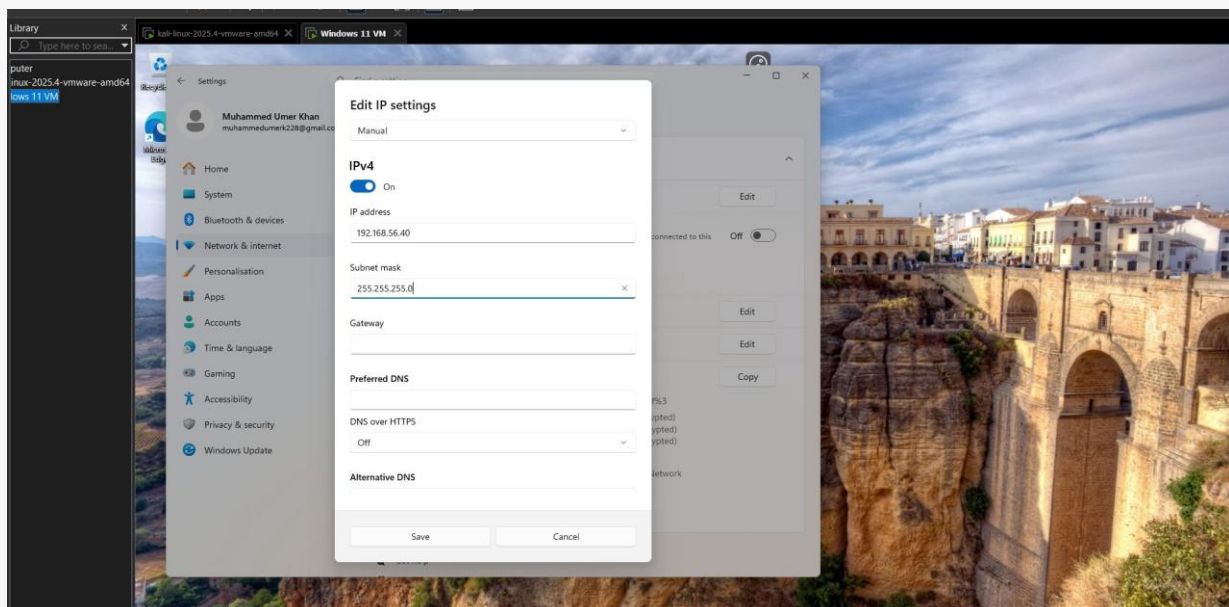


Figure 6 — Windows 11: Edit IP settings showing Manual IPv4 set to 192.168.56.40 / 255.255.255.0



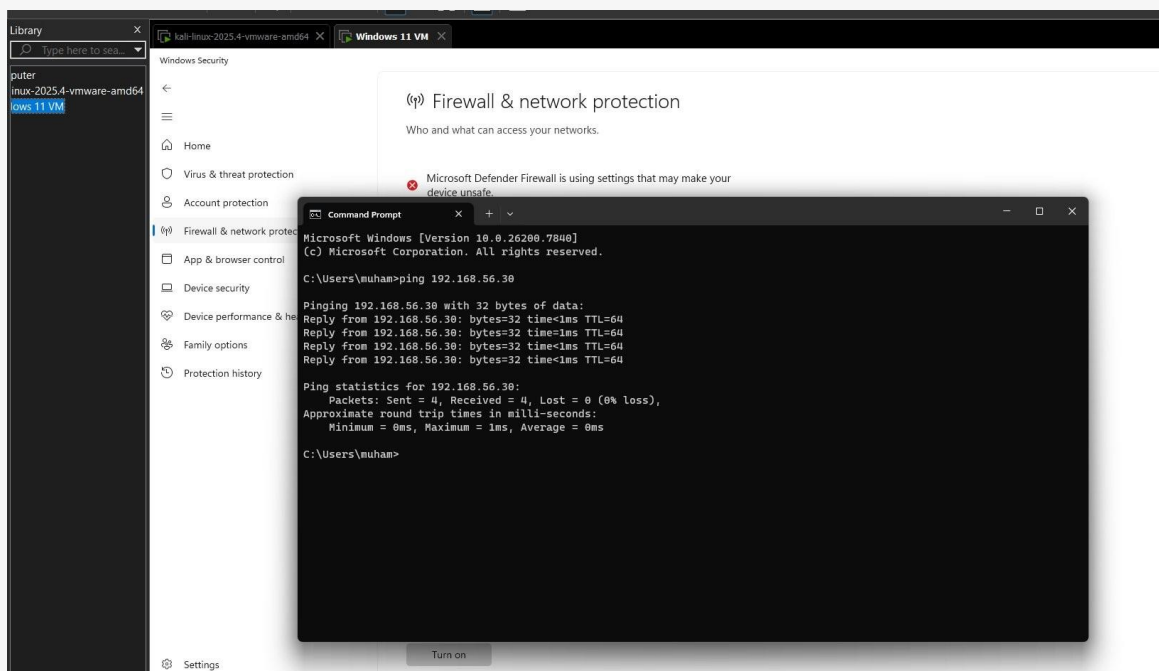


Figure 7 — Windows Defender Firewall disabled across all profiles to allow ICMP traffic

## 1.4 Bidirectional Connectivity Verification

Both machines were pinged from each other to confirm the LAN Segment is operational. From Windows 11: ping 192.168.56.30 returned 4 packets sent, 4 received, 0% loss, avg 0ms. From Kali Linux: ping 192.168.56.40 returned 6 packets sent, 6 received, 0% loss.

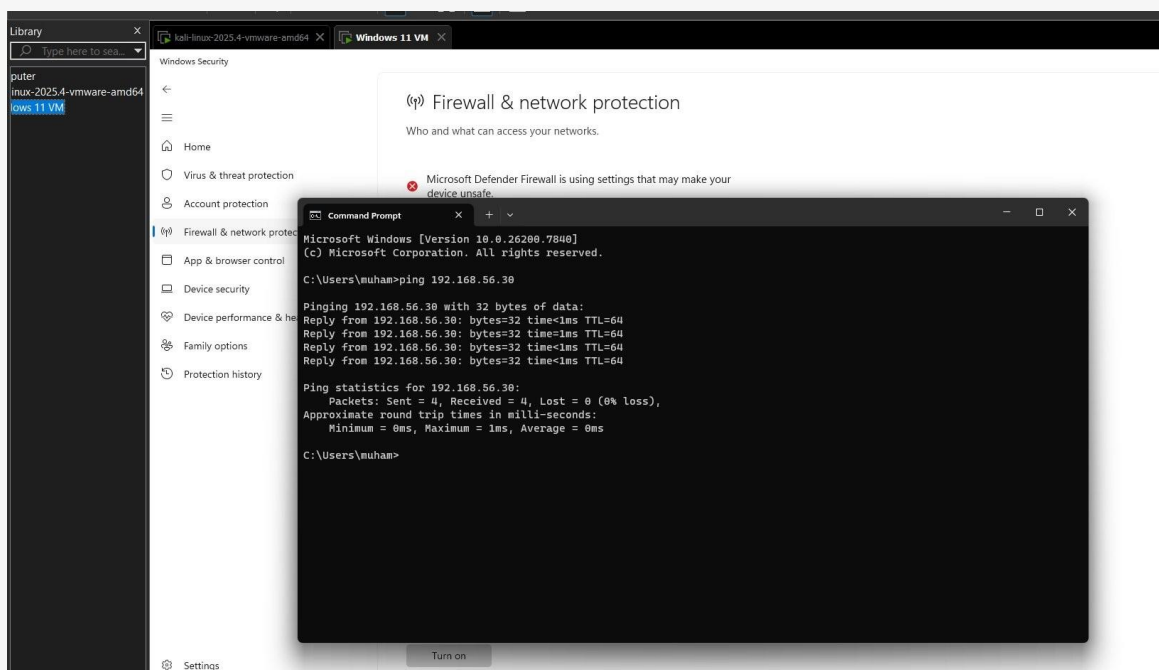


Figure 8 — Windows 11 → Kali (192.168.56.30): 4/4 packets received, 0% loss, <1ms latency

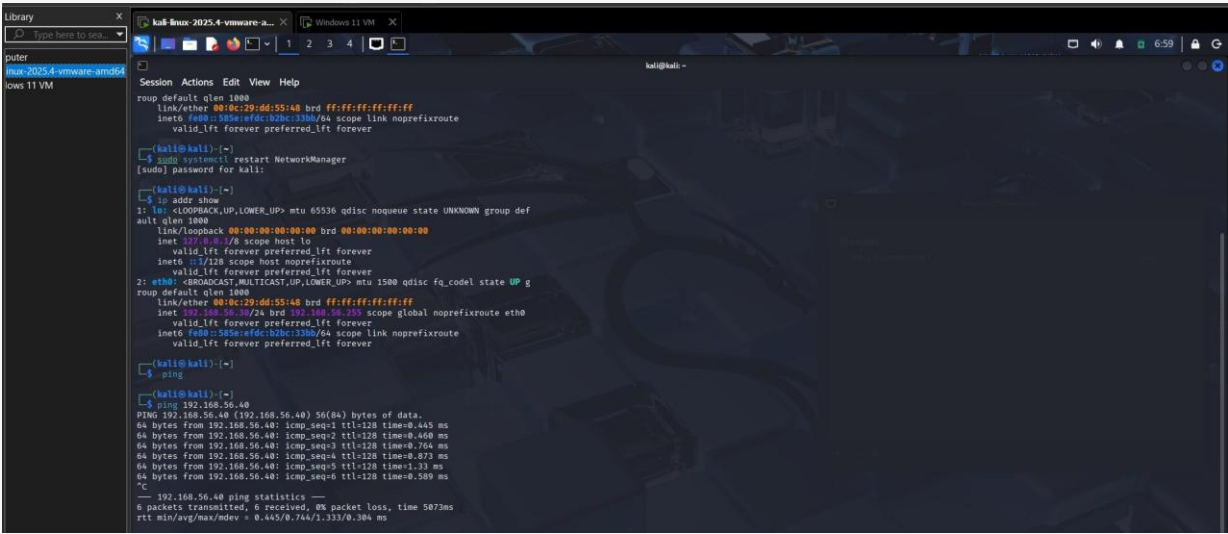


Figure 9 — Kali Linux → Windows 11 (192.168.56.40): 6/6 packets received, 0% packet loss

1.5 External Internet Isolation Verification

To confirm the lab has zero internet connectivity, a ping was sent to Google’s public DNS (8.8.8.8) from Windows 11. All 4 packets returned “PING: transmit failed. General failure.” with 100% packet loss. This confirms the LAN Segment provides complete isolation from the external internet.

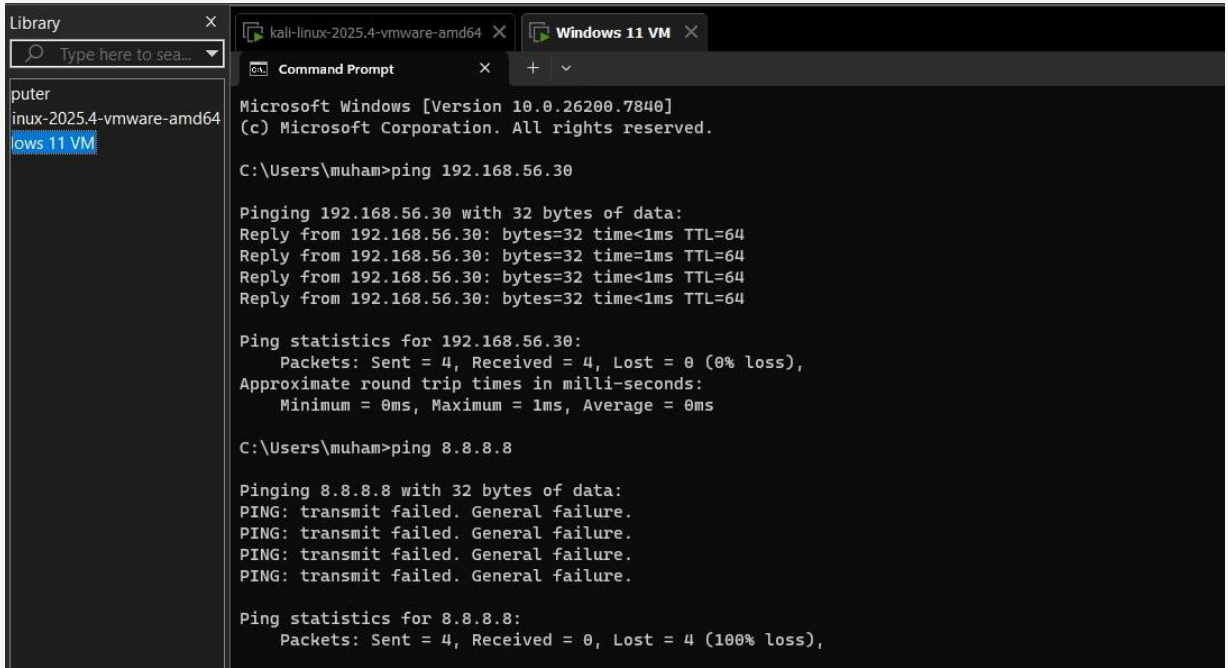


Figure 10 — ping 8.8.8.8 from Windows 11 returns 100% packet loss — internet isolation confirmed

Task 01 — Completion Status

| Requirement | Status |
|-------------|--------|
|-------------|--------|

|                                                                  |            |
|------------------------------------------------------------------|------------|
| VMware LAN Segment "Intnet" created and applied to both VMs      | ✓ Complete |
| Kali Linux static IP 192.168.56.30/24 assigned on eth0           | ✓ Complete |
| Windows 11 static IP 192.168.56.40/24 configured manually        | ✓ Complete |
| Windows Defender Firewall disabled for ICMP testing              | ✓ Complete |
| Bidirectional ping successful (Windows 11 ↔ Kali Linux)          | ✓ Complete |
| External internet isolation confirmed (ping 8.8.8.8 = 100% loss) | ✓ Complete |

■ Task 02 — Packet Analysis & Protocol Deep-Dive

■ Objective

To use Wireshark on Kali Linux to capture live network traffic and analyze the TCP Three-Way Handshake in detail — identifying SYN, SYN-ACK, and RST packets, and examining the Ethernet (Layer 2), IP (Layer 3), and TCP (Layer 4) headers for their forensic significance.

■ 2.1 Methodology

- Wireshark launched on Kali: `sudo wireshark` — capture started on interface `eth0`
- Attempted `smbclient -L //192.168.56.40` to generate SMB traffic — returned a usage/syntax error
- Switched to Nmap: `nmap -p 445 192.168.56.40` — successfully generated TCP SYN traffic to port 445
- Wireshark filter `tcp.port == 445` applied — three handshake packets isolated and analyzed

■ 2.2 Wireshark Live Capture

Wireshark was started on `eth0` while the ping tests were still running. Initial captures showed ICMP traffic and a Windows Browser Protocol broadcast — confirming `eth0` was actively receiving traffic from the Windows 11 VM.



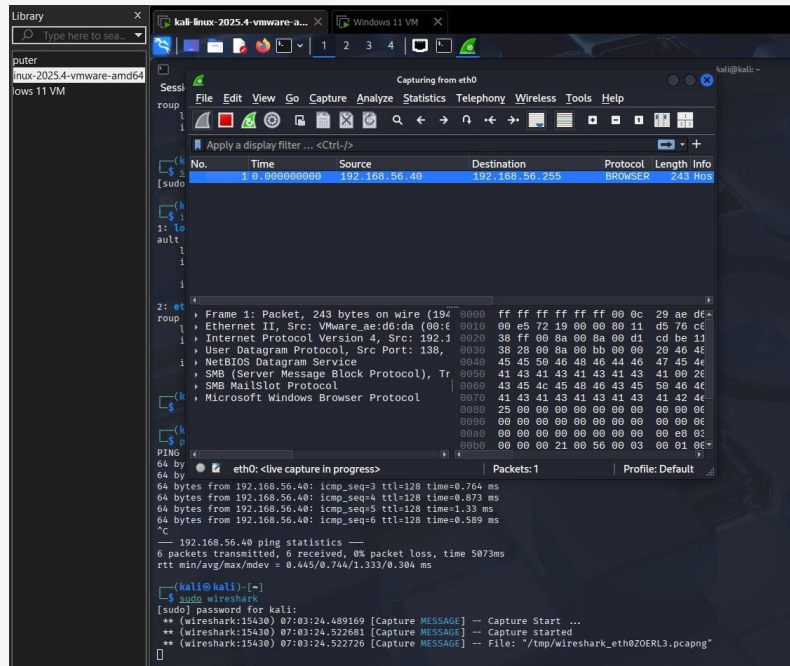


Figure 11 — Wireshark: live capture started on eth0, showing active traffic from Windows 11 VM

## 2.3 SMBclient Syntax Issue & Switch to Nmap

The first attempt — `smbclient -l//192.168.56.40` — produced a usage error because of a missing space between `-l` and `//`. Rather than troubleshoot the SMB syntax, Nmap was used as an alternative to generate TCP port 445 traffic, which produces an equivalent and cleaner TCP handshake for analysis.

```

(kali@kali)-[~/Desktop]
$ smbclient -l//192.168.56.40
[2026/03/08 07:27:46, 0] lib/util/debug.c:1269(reopen_one_log)
  reopen_one_log: Unable to open new log file '//192.168.56.40/log.smbclient'
: No such file or directory
Usage: smbclient [-?EgqBNPKV] [-?|-help] [--usage] [-M|-message=HOST]
  [-I|-ip-address=IP] [-E|-stderr] [-L|-list=HOST]
  [-T|-tar=<c|x>IXFvgbNan] [-D|-directory=DIR] [-c|-command=STRING]
  [-b|-send-buffer=BYTES] [-t|-timeout=SECONDS] [-p|-port=PORT]
  [-g|-grepable] [-q|-quiet] [-B|-browse]
  [-d|-debuglevel=DEBUGLEVEL] [--debug-stdout]
  [-s|-configfile=CONFIGFILE] [--option=name=value]
  [-l|-log-basename=LOGFILEBASE] [--leak-report] [--leak-report-full]
  [-R|-name-resolve=NAME-RESOLVE-ORDER]
  [-O|-socket-options=SOCKETOPTIONS] [-m|-max-protocol=MAXPROTOCOL]
  [-n|-netbiosname=NETBIOSNAME] [--netbios-scope=SCOPE]
  [-W|-workgroup=WORKGROUP] [--realm=REALM]
  [-U|-user=[DOMAIN/]USERNAME[%PASSWORD]] [-N|-no-pass]
  [--password=STRING] [--pw-nt-hash] [-A|-authentication-file=FILE]
  [-P|-machine-pass] [--simple-bind-dn=DN]
  [--use-kerberos=desired|required|off] [--use-krb5-ccache=CCACHE]
  [--use-winbind-ccache] [--client-protection=sign|encrypt|off]
  [-k|-kerberos] [-V|-version] [OPTIONS] service <password>

```

Figure 12 — smbclient syntax error (missing space) — Nmap used as the alternative tool

## 2.4 Nmap Port 445 Scan

Nmap confirmed TCP port 445 (microsoft-ds / SMB) is open on the Windows 11 target. The scan completed in 13.45 seconds with a response latency of 0.0018 seconds, and identified the NIC as VMware (MAC: 00:0C:29:AE:D6:DA).

```

(kali@kali)-[~/Desktop]
$ smbclient -l //192.168.56.40
2026/03/08 07:29:07, 0] lib/util/debug.c:1269(reopen_one_log)
  reopen_one_log: Unable to open new log file '//192.168.56.40/log.smbclient'
  No such file or directory
Usage: smbclient [-?EgqBNPKV] [-?|-help] [--usage] [-M|-message=HOST]
  [-I|-ip-address=IP] [-E|-stderr] [-L|-list=HOST]
  [-T|-tar=<c|x>IXFvgbNan] [-D|-directory=DIR] [-c|-command=STRING]
  [-b|-send-buffer=BYTES] [-t|-timeout=SECONDS] [-p|-port=PORT]
  [-g|-grepable] [-q|-quiet] [-B|-browse]
  [-d|-debuglevel=DEBUGLEVEL] [--debug-stdout]
  [-s|-configfile=CONFIGFILE] [--option=name=value]
  [-l|-log-basename=LOGFILEBASE] [--leak-report] [--leak-report-full]
  [-R|-name-resolve=NAME-RESOLVE-ORDER]
  [-O|-socket-options=SOCKETOPTIONS] [-m|-max-protocol=MAXPROTOCOL]
  [-n|-netbiosname=NETBIOSNAME] [--netbios-scope=SCOPE]
  [-W|-workgroup=WORKGROUP] [--realm=REALM]
  [-U|-user=[DOMAIN/]USERNAME[%PASSWORD]] [-N|-no-pass]
  [--password=STRING] [--pw-nt-hash] [-A|-authentication-file=FILE]
  [-P|-machine-pass] [--simple-bind-dn=DN]
  [--use-kerberos=desired|required|off] [--use-krb5-ccache=CCACHE]
  [--use-winbind-ccache] [--client-protection=sign|encrypt|off]
  [-k|-kerberos] [-V|-version] [OPTIONS] service <password>

(kali@kali)-[~/Desktop]
$ nmap -p 445 192.168.56.40
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-08 07:30 EDT

```

Figure 13 — Nmap starting: nmap -p 445 192.168.56.40 executed from Kali terminal

```
kali@kali: ~/Desktop
Session Actions Edit View Help
[-d|--debuglevel=DEBUGLEVEL] [--debug-stdout]
[-s|--configfile=CONFIGFILE] [--option=name=value]
[-l|--log-basename=LOGFILEBASE] [--leak-report] [--leak-report-full]
[-R|--name-resolve=NAME-RESOLVE-ORDER]
[-O|--socket-options=SOCKETOPTIONS] [-m|--max-protocol=MAXPROTOCOL]
[-n|--netbiosname=NETBIOSNAME] [--netbios-scope=SCOPE]
[-W|--workgroup=WORKGROUP] [--realm=REALM]
[-U|--user=[DOMAIN/]USERNAME[%PASSWORD]] [-N|--no-pass]
[--password=STRING] [--pw-nt-hash] [-A|--authentication-file=FILE]
[-P|--machine-pass] [--simple-bind-dn=DN]
[--use-kerberos=desired|required|off] [--use-krb5-ccache=CCACHE]
[--use-winbind-ccache] [--client-protection=sign|encrypt|off]
[-k|--kerberos] [-V|--version] [OPTIONS] service <password>

(kali@kali)-[~/Desktop]
$ nmap -p 445 192.168.56.40
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-08 07:30 EDT
Nmap scan report for 192.168.56.40
Host is up (0.0018s latency).

PORT      STATE SERVICE
445/tcp   open  microsoft-ds
MAC Address: 00:0C:29:AE:D6:DA (VMware)

Nmap done: 1 IP address (1 host up) scanned in 13.45 seconds
```

Figure 14 — Nmap result: port 445/tcp OPEN on Windows 11, MAC confirmed VMware NIC

## 2.5 TCP Three-Way Handshake — Wireshark Capture

After applying `tcp.port == 445` filter, three packets were isolated. Packet #35 is the SYN (Kali initiating), Packet #36 is the SYN-ACK (Windows 11 accepting), and Packet #37 is RST (Nmap resetting the connection — normal behavior for a port scan that does not complete an application-level session).

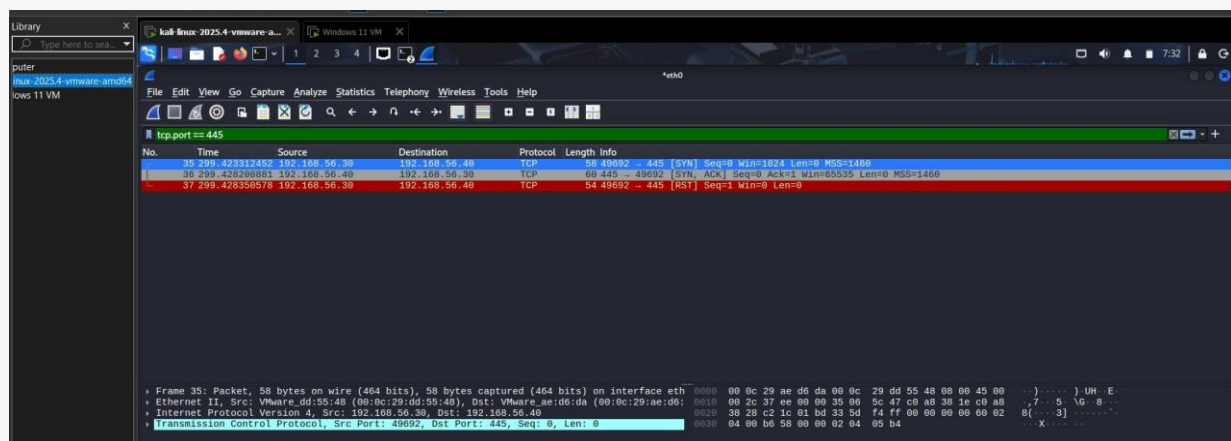


Figure 15 — Wireshark: tcp.port == 445 filter — SYN (#35), SYN-ACK (#36), RST (#37) packets visible

| Packet | Flag       | Source                | Destination           | Meaning               |
|--------|------------|-----------------------|-----------------------|-----------------------|
| #35    | [SYN]      | 192.168.56.30 (Kali)  | 192.168.56.40 (Win11) | Connection initiation |
| #36    | [SYN, ACK] | 192.168.56.40 (Win11) | 192.168.56.30 (Kali)  | Connection accepted   |

|     |       |                      |                       |                     |
|-----|-------|----------------------|-----------------------|---------------------|
| #37 | [RST] | 192.168.56.30 (Kali) | 192.168.56.40 (Win11) | Nmap reset (normal) |
|-----|-------|----------------------|-----------------------|---------------------|

## 2.6 Ethernet Header Analysis (OSI Layer 2)

Expanding the Ethernet II header reveals physical addressing. Source MAC `00:0c:29:dd:55:48` is Kali's VMware NIC. Destination MAC `00:0c:29:ae:d6:da` is Windows 11's VMware NIC. EtherType `0x0800` confirms the payload is an IPv4 datagram.

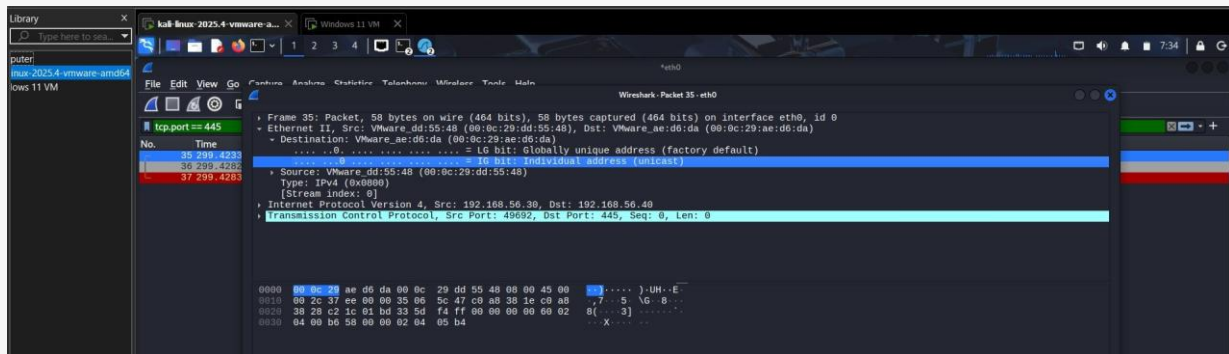


Figure 16 — Wireshark: Ethernet II header expanded — Source & Destination MAC addresses, Type = IPv4

## 2.7 IP Header Analysis (OSI Layer 3)

The IPv4 header provides logical addressing. Source IP `192.168.56.30` (Kali) is initiating the connection to Destination IP `192.168.56.40` (Windows 11). Protocol field 6 confirms TCP is the transport layer protocol. TTL of 53 indicates remaining hop count and can assist in OS fingerprinting.

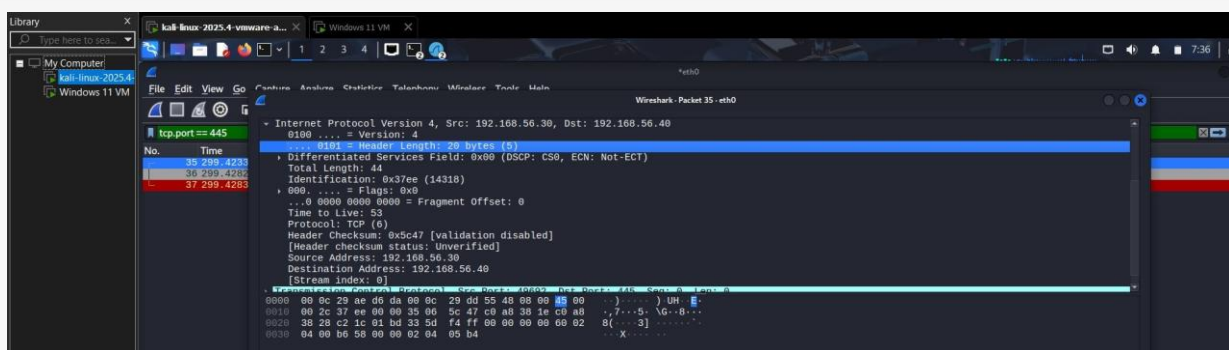


Figure 17 — Wireshark: IPv4 header expanded — Src IP, Dst IP, Protocol TCP (6), TTL = 53

## 2.8 TCP Header Analysis (OSI Layer 4)

The TCP header contains the richest forensic data. Source Port `49692` is Kali's dynamic ephemeral port. Destination Port `445` is the Windows SMB service. Flags field `0x002` confirms this is the SYN packet. Sequence Number `0` (relative), Window Size `1024`, and MSS `1460` are also visible — providing insight into the sender's buffer capacity.



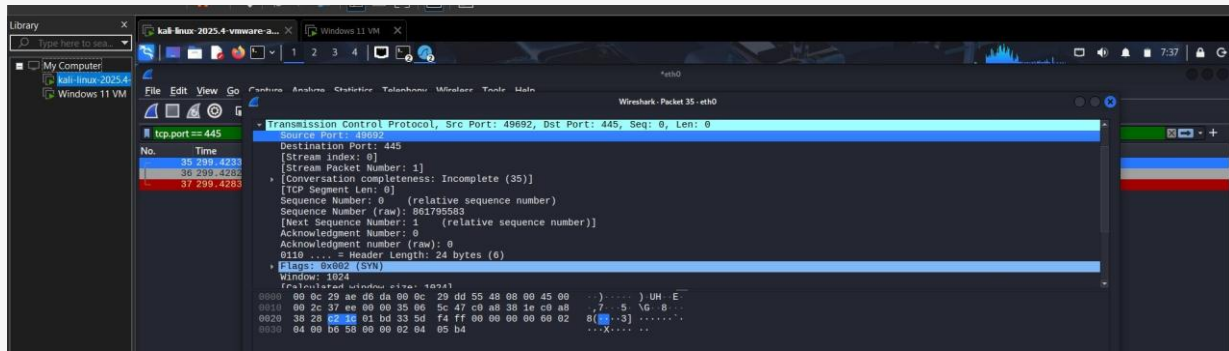


Figure 18 — Wireshark: TCP header expanded — Src Port 49692, Dst Port 445, SYN flag, Window 1024

## 2.9 Protocol Header Forensic Summary

| Layer    | Field            | Value             | Forensic Significance                  |
|----------|------------------|-------------------|----------------------------------------|
| Ethernet | Source MAC       | 00:0c:29:dd:55:48 | Identifies Kali Linux VMware NIC       |
| Ethernet | Destination MAC  | 00:0c:29:ae:d6:da | Identifies Windows 11 VMware NIC       |
| Ethernet | EtherType        | 0x0800 (IPv4)     | Confirms Layer 3 protocol in payload   |
| IP       | Source IP        | 192.168.56.30     | Kali Linux — connection initiator      |
| IP       | Destination IP   | 192.168.56.40     | Windows 11 — connection target         |
| IP       | Protocol         | 6 (TCP)           | Transport layer protocol identifier    |
| IP       | TTL              | 53                | Hop count — assists OS fingerprinting  |
| TCP      | Source Port      | 49692             | Dynamic ephemeral port (Kali)          |
| TCP      | Destination Port | 445               | Windows SMB / file sharing service     |
| TCP      | Flags            | 0x002 (SYN)       | Marks this as connection initiation    |
| TCP      | Window Size      | 1024              | Receive buffer — flow control hint     |
| TCP      | MSS              | 1460              | Maximum segment size for fragmentation |

## Task 02 — Completion Status

| Requirement                                                 | Status     |
|-------------------------------------------------------------|------------|
| Wireshark launched on Kali Linux, capture started on eth0   | ✓ Complete |
| TCP traffic generated via Nmap scan on port 445             | ✓ Complete |
| SYN (#35) and SYN-ACK (#36) packets identified in Wireshark | ✓ Complete |
| Ethernet (Layer 2) header expanded and documented           | ✓ Complete |
| IP (Layer 3) header expanded and documented                 | ✓ Complete |
| TCP (Layer 4) header expanded and documented                | ✓ Complete |
| Forensic significance of all header fields analyzed         | ✓ Complete |



## ■ Key Learnings & Observations

### Network Isolation in Practice

Building the LAN Segment showed exactly how organizations create isolated security labs. Without DHCP or a gateway, every IP must be manually planned — reinforcing IP address management as a foundational skill. The isolation verification (ping 8.8.8.8 → 100% loss) proved that no traffic can leak to the internet, which is critical for safe malware analysis and penetration testing environments.

### TCP Handshake as Forensic Evidence

Watching the SYN → SYN-ACK → RST sequence live in Wireshark made the textbook concept tangible. Every TCP connection leaves this trail. From just 3 packets, an analyst can identify: which machine initiated the connection, what service was targeted, the buffer capacity of the sender, an estimate of the remote OS (via TTL), and the exact timestamp of each exchange.

### Real-World Troubleshooting

Two issues arose during Task 02: (1) smbclient had a syntax error due to a missing space between `-L` and `//` — resolved by switching to Nmap which generated equivalent TCP traffic more reliably. (2) An initial Wireshark capture showed no filtered packets because the capture was started before Nmap ran — restarting after the scan was complete resolved this. These situations reflect real-world scenarios where tools behave unexpectedly and the ability to pivot quickly is essential.

## ■ Conclusion

Week 1 of the Cyberster Purple Team Internship successfully delivered two foundational competencies: isolated virtual lab construction and live network packet analysis.

The VMware lab with Kali Linux and Windows 11 on a private LAN Segment is fully operational and correctly isolated. The Wireshark exercise proved that each TCP/IP layer carries distinct forensic fingerprints — Ethernet for physical identity, IP for logical addressing, and TCP for connection intent and state. These skills underpin both offensive security (understanding what traffic attacks generate) and defensive security (detecting and interpreting that traffic in real time).

The controlled lab environment built this week serves as the permanent foundation for all upcoming Purple Team exercises in Weeks 2 and 3.